# Deterministic decremental SSSP and approximate min cost flow in almost linear time

Donghyeon Lee

# Single source shortest paths

## Static SSSP

Input is an undirected weighted graph $G = (V, E, w)$ and a source $s \in V$.

$$\text{Goal: compute } \mathrm{dist}_G(s, v) \text{ for all } v \in V.$$

Classic algorithms run in near linear time in $m = |E|$.

## Two outputs we care about

- Distance estimates for all vertices
- A short representation of an approximate path $s \rightsquigarrow v$

# Dynamic shortest paths

## Dynamic algorithm viewpoint

Graph changes over time and we want to maintain information without recomputing from scratch.

## Fully dynamic SSSP

- Updates can insert or delete edges and change weights
- Queries ask for $\mathrm{dist}(s, v)$ or a path
- Conditional lower bounds essentially rule out fully dynamic algorithms with small update *and* query times for maintaining distances from $s$

## Decremental SSSP

- Updates are *edge deletions*
- Distances are monotone nondecreasing
- This model admits much faster *total* update time

# Adversarial models

### Oblivious (non-adaptive) adversary

The entire sequence of updates and queries is fixed in advance. Random choices can remain hidden.

### Adaptive adversary

Next update can depend on previous answers. Example: query a path, then delete all edges on that path.

### Why this matters here

Fast decremental SSSP is needed as a black box inside MWU where updates depend on returned paths. So we need deterministic (or otherwise adaptive-adversary) guarantees.

# Why oblivious adversary guarantees are often unusable

## Two common failure modes

- **Real feedback loops:** in networking, routing, or control, the environment responds to your decisions, i.e., updates are inherently adaptive.
- **Black-box composition:** if an algorithm is used inside another algorithm, the outer routine may choose updates based on the inner routine's answers.

## Concrete example

MWU repeatedly asks for a shortest path $\pi$ and then changes weights on the returned path. This is an adaptive update rule.

## So

To be a reliable primitive, decremental SSSP must tolerate adaptivity, which is why determinism is so valuable here.

# State of the art before this paper

## Unweighted case

Even and Shiloach maintain exact distances with total update time $O(mn)$. This is near optimal under conditional lower bounds.

## Weighted and approximate case

For $(1+\varepsilon)$-approximation in undirected weighted graphs, the breakthrough of Forster–Henzinger–Nanongkai (FOCS'14) achieves essentially optimal total update time $\widehat{O}(m \cdot \mathrm{polylog}\, W)$, but it is randomized and only works against an oblivious adversary.

## Best known adaptive/deterministic bounds

Despite major progress, the best adaptive guarantees still had polynomial overhead, around $\widehat{O}(\min\{m\sqrt{n},\, n^2\} \cdot \mathrm{polylog}\, W)$.

# The adaptive gap that this paper closes

## Same problem, different adversary assumptions

| Setting | Guarantee | Total update time |
|---|---|---|
| Randomized, non-adaptive | $(1+\varepsilon)$ distances | $\widehat{O}(m)$ up to subpoly factors |
| Deterministic / adaptive (older) | $(1+\varepsilon)$ distances | $\widehat{O}(\min\{m\sqrt{n}, n^2\})$ (subpoly/log factors) |
| **This paper** | $(1+\varepsilon)$ distances | $\mathbf{m^{1+o(1)}}\mathrm{polylog}\mathbf{W}$ |

# Main results

**Theorem 1 (Theorem I.1.1 (decremental SSSP))**

*Given an undirected weighted graph $G = (V, E, w)$ undergoing edge deletions, a fixed source $s$, and $\varepsilon > 1/\mathrm{polylog} n$, there is a deterministic data structure that maintains $(1+\varepsilon)$-approximate distance estimates explicitly for all vertices with total update time*

$$m^{1+o(1)} \mathrm{polylog}\, W,$$

*where $W$ is the maximum weight ratio. Moreover, it supports $(1+\varepsilon)$-approximate path queries in time $|\pi(s,t)| \cdot n^{o(1)}$.*

# Main results

**Theorem 2 (Theorem I.1.2 (approx. mixed-capacitated min-cost flow))**

*There is a randomized algorithm running in time*

$$m^{1+o(1)} \log \log C$$

*that, with high probability, returns a feasible flow sending a* $(1-\varepsilon)$*-fraction of the maximum s-t flow value with cost at most the min-cost (max) flow cost. Here* $C$ *denotes the max–min ratio of costs.*

# A paper in three parts

### Part II

Distance-only decremental SSSP data structure. This is the central technical contribution and is largely self-contained.

### Part III

Extension to *path reporting* and *threshold subpath queries*. Needed to beat the flow decomposition barrier inside MWU.

### Part IV

Approximate min cost flow using a modified MWU framework. This part treats the SSSP structure as a black box.

### Today

We follow Part I overview and then dive into the distance-only design from Part II.

# Flow problems in scope

## Capacities

- **Edge capacities:** $u(e) \geq 0$ and feasible flow satisfies $f(e) \leq u(e)$.
- **Vertex capacities:** $u(v) \geq 0$ and feasible flow satisfies $\mathrm{inflow}(v) \leq u(v)$ (equivalently, total flow through $v$ is bounded).

## Costs

Assign nonnegative costs $c$ (on edges and/or vertices). The cost of a flow is typically

$$\mathrm{cost}(f) = \sum_{e \in E} c(e)\, f(e) \;+\; \sum_{v \in V} c(v)\, \mathrm{inflow}(v).$$

## Two common optimization views

- **Bounded-cost max flow:** maximize flow value subject to $\mathrm{cost}(f) \leq B$.
- **Min-cost max flow:** among max-value flows, minimize $\mathrm{cost}(f)$.

# Where approximate undirected flow stood

## Previous results

- **Undirected, edge-capacitated, approximate:** near-linear time was known.
- **Vertex capacities and/or finite-capacity min-cost:** best known bounds incurred an additional polynomial term $n^{3/2}$.

## This paper's flow contribution

A new MWU-style framework together with a strong decremental SSSP oracle yields almost-linear time approximation for *min-cost flow with costs/capacities on both vertices and edges*.

# Why shortest paths help flow algorithms

## Multiplicative weight update viewpoint

Many undirected flow problems can be solved by iteratively routing along approximate shortest paths in a changing metric. Each iteration updates weights on a path and recomputes a shortest path in the new metric.

## Key connection

When weights only increase, the sequence of shortest path computations can be accelerated by a decremental SSSP data structure. This is why the decremental model is so well motivated.

# MWU framework in its simplest form

## High level algorithm

```
Initialize flow f ← 0
Initialize weights w ← w_0 on vertices or edges
while true do
    Compute a (1 + ε)-approximate s-t path π under weights w
    if w(π) is too large then
        break
    end if
    Send λ(π) units of flow along π
    Increase weights on objects of π multiplicatively
end while
Output scaled version of f
```

## What shortest path oracle must handle

- Metric updates depend on returned paths
- So updates are adaptive
- Weights only increase so decremental SSSP is enough

## What makes it fast

- Replace recomputation with dynamic updates
- Reduce total time to total update time of decremental SSSP plus path update overhead

# Two challenges of MWU

### Goal

Execute the MWU framework in almost linear time in $m$.

### First challenge

Need an adaptive-adversary decremental SSSP algorithm with almost linear total update time. This was a long standing open problem.

### Second challenge

Total length of all routed paths can be huge. Updating every edge of every path can dominate the running time. This is known as the flow decomposition barrier.

# Flow decomposition barrier

## Question

Given an $s$-$t$ flow, decompose it into weighted $s$-$t$ paths. How large can the total path length $\sum_p |p|$ be in the worst case?

## Worst case bounds

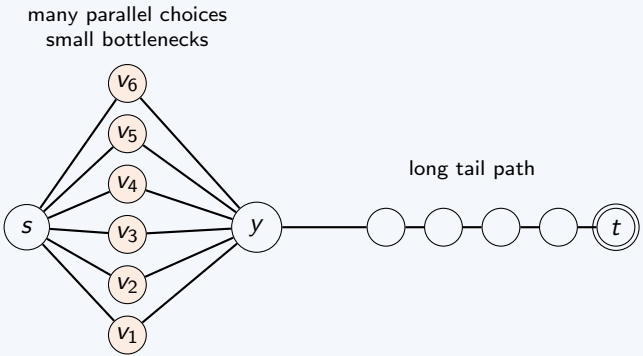
- Unit edge capacities: $O(m \log n)$
- General edge capacities: $\Omega(mn \log n)$
- General vertex capacities: $\Theta(n^2 \log n)$

## Implication for MWU

Standard MWU routes flow along entire paths and updates every edge or vertex on the path. So even with fast decremental SSSP, the path update cost can be superlinear.

# A canonical barrier example

## Graph gadget




## Intuition

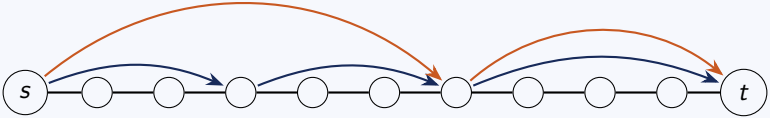
Each MWU iteration may select a path through one of the $v_i$ and then traverse the long tail. Each path pushes only a tiny amount of flow because of the bottleneck. So MWU may use many long paths before saturating the gadget.

# Hop compression and layering

## Idea in one sentence

Maintain a sparse emulator $H$ of $G$ so that long shortest paths in $G$ become short-hop paths in $H$. Then recurse to compress hops again.

## Layering picture




black edges are $G$
blue edges are $H_1$
orange edges are $H_2$

# Hop emulators

## Emulator definition

A weighted graph $H$ on a superset of vertices is an $(h, \varepsilon)$ emulator for $G$ if:

- for all $u, v \in V(G)$: $\mathrm{dist}_G(u, v) \leq \mathrm{dist}_H(u, v)$,
- for all $u, v \in V(G)$: there exists a path in $H$ with at most $h$ edges and total weight at most $(1 + \varepsilon)\mathrm{dist}_G(u, v)$.

## What we want at one layer

Given $G$ and hop parameter $\gamma = n^{o(1)}$, maintain an emulator that reduces hop length by about $\gamma$ while keeping only $\widetilde{O}(m)$ edges and $\widehat{O}(m)$ total maintenance time at that layer.

# Covering as the structure behind the first emulator

## Covering in the overview

We maintain a collection of cores $C_1, \ldots, C_q$ with a core scale $d$ and a larger shell scale $D$, where $d, D$, and $D/d$ are all $n^{o(1)}$.

## Idealized properties

1. Each core is monotone and can only lose vertices over time
2. Each core has weak diameter at most about $d$
3. Every vertex stays close to some core: for every $v \in V$ there exists $i$ such that $v \in \mathrm{ball}(C_i, 4d)$
4. Over the whole update sequence, each vertex lies in at most $n^{o(1)}$ shells of the form $\mathrm{ball}(C_i, D)$

# Why covering yields hop compression

## Emulator construction from covering

Introduce a core vertex $v_{C_i}$ for each core $C_i$.

- For $v \in C_i$, add an edge $(v, v_{C_i})$ of weight $d$
- For $w \in \mathrm{ball}(C_i, D)$, add an edge $(w, v_{C_i})$ of weight $\mathrm{dist}_G(w, C_i)$

Edges appear and disappear as vertices enter or leave shells.

## Two hop shortcut

If $\mathrm{dist}_G(u, v) \approx D/2$, $u$ is near $C_i$, and $v$ lies in the shell of $C_i$, then

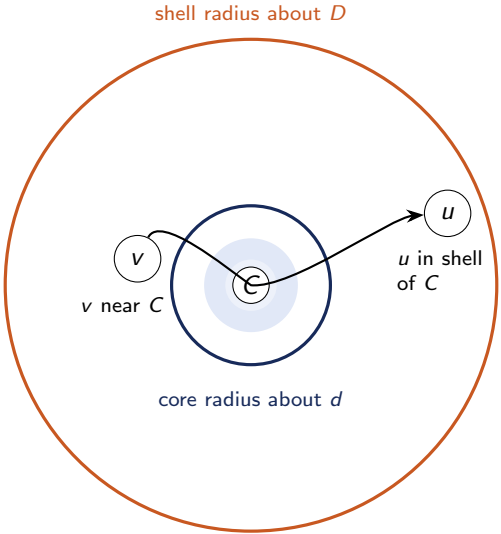
$$u \to v_{C_i} \to v$$

uses two edges and approximates that segment in $G$.

## Work bound intuition

Property four bounds the total size of maintained shells and thus the total work across all local ES/MES structures.

# A geometric picture of cores and shells



# Robust core

## Simplified RobustCore goal

Assume we start from a graph region whose weak diameter is small, say

$$\mathrm{diam}_G(K_{\mathrm{init}}) \leq d \qquad (d = n^{o(1)}).$$

Under edge deletions, maintain a subset $K \subseteq K_{\mathrm{init}}$ such that:

- **(Small diameter)** the surviving set stays "ball-like": $\mathrm{diam}_G(K) \leq \widehat{O}(d)$.
- **(Scattering / controlled shrink)** when a vertex $v$ is removed from the core, the local terminal mass around $v$ has noticeably dropped.
- **(Termination)** once $K$ becomes much smaller than $K_{\mathrm{init}}$, we are allowed to stop and restart on what remains.

# Robust core

## Robust core in the overview (Definition I.2.4)

Input is a decremental graph $G$ (edge deletions) and an initial set $K_{\text{init}} \subseteq V(G)$ with small weak diameter

$$\operatorname{diam}_G(K_{\text{init}}) \leq d \qquad (d = n^{o(1)}).$$

Maintain a decremental set $K \subseteq K_{\text{init}}$ until $K = \emptyset$ such that

- Scattering property: for every removed vertex $v \in K_{\text{init}} \setminus K$,

  $$\left|\operatorname{ball}_G(v, 2d) \cap K_{\text{init}}\right| \leq (1 - \delta_{\text{scatter}}) \left|K_{\text{init}}\right|$$

  for $\delta_{\text{scatter}} = 1/n^{o(1)}$.

- Diameter property:

  $$\operatorname{diam}_G(K) \leq \mathrm{str} \cdot d$$

  for $\mathrm{str} = n^{o(1)}$.

# Why previous approaches stumble on adaptivity

## Random center trick for oblivious adversaries

Pick a random $k \in K_{\text{init}}$ and maintain $\text{ball}(k, 7d) \cap K_{\text{init}}$ with an ES tree. If $|\text{ball}(k, 4d) \cap K_{\text{init}}|$ becomes too small, resample a new center.

## Why it works when adversary is fixed

A uniformly random center is likely to be in a well connected region. If its ball shrinks, then many vertices have shrinking neighborhoods, so scattering removes many vertices at once.

## Why it fails for an adaptive adversary

The adversary can infer the sampled center through queries and delete edges to isolate it while keeping the rest well connected. Randomness is no longer independent from updates.

# RobustCore is easy on a expander

## Idealized scenario

Suppose the current graph on terminals behaves like a vertex expander. Then it is hard for deletions to separate a large terminal subset without deleting a lot of edges.

## Conceptual strategy

- Maintain a large expander subset $X \subseteq K$ using expander pruning.
- Maintain a multi-source shortest-path tree (or ball) from $X$ out to radius $\Theta(d)$.
- If $v$ leaves that ball, it is no longer close to the "well-connected center" and is removed from $K$.

## Why this is informative

This is the deterministic analogue of the "random center" idea: instead of rooting at a random vertex, root at a *maintained expander core*.

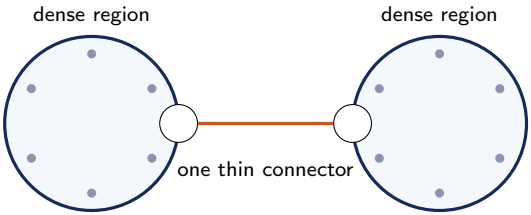
## Where it breaks

General low-diameter graphs may contain thin bottlenecks (not expanders), so we need a way to *enforce* expansion.

# If the graph is not an expander

## Two extremes

- In a strong expander: no single vertex/edge can isolate many terminals easily.
- In an arbitrary graph: there may be many very fragile connectors.
- In a *small-diameter* graph: the total amount of fragility is limited, and we can quantify it with a capacity assignment.




## Next idea

Assign vertex capacities so that separating many terminals requires paying a lot of capacity.

# Expander based dynamic algorithms and a common bottleneck

## Expander tools

Many dynamic graph problems are easier on expanders because we can use expander pruning and expander embeddings.

## Expander decomposition limitation

To handle general graphs, many works use expander decomposition and then process crossing edges that do not belong to any expander piece. For decremental shortest paths, the number of crossing edges becomes a major obstacle to near-linear total update time.

## Key new idea of this paper

Instead of decomposing into expanders, assign vertex capacities $\kappa(v)$ to make the graph a *capacitated* vertex expander with respect to the terminal set. Then apply expander tools directly.

# Expander reminder

## Uncapacitated vertex expansion

A graph is a vertex expander if every small set $L$ has many neighbors $N(L)$ outside $L$. In constant degree graphs this implies small diameter and robust connectivity.

## Expander pruning

Given a decremental expander, there are data structures that maintain a large expander subset $X$ while edges are deleted, until too much edge mass is removed.

## Need a weighted generalization

In low diameter graphs, unweighted expansion may be absent. We will enforce an expansion condition by assigning vertex capacities $\kappa(v)$.

# Capacitated vertex expansion

## Capacitated cut

Consider a cut $L, S, R$ where $S$ separates $L$ from $R$. The cost of the cut is the capacity $\kappa(S) = \sum_{v \in S} \kappa(v)$.

## Capacitated expander definition

For a terminal set $K$, a cut $L, S, R$ is a *sparse capacitated vertex cut* if

$$\kappa(S) \leq \frac{|L \cap K|}{n^{o(1)}}.$$

We say $(K, \kappa)$ forms a capacitated expander if no such sparse cut exists.

## Interpretation

Vertices in the separator must "pay" for separating many terminals. If a cut is too cheap, we increase capacities near it until it is no longer cheap.

# Key structural lemma for low diameter graphs

**Theorem 3 (Lemma I.2.9 (Small capacity sum for small diameter))**

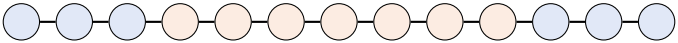
*Given graph $G$ and any $K \subseteq V(G)$, there exists a capacity function $\kappa$ such that $(K, \kappa)$ forms a capacitated expander in $G$ and*

$$\sum_{v \in V(G)} \kappa(v) = \widetilde{O}(|K| \mathrm{diam}_G(K)).$$

**Tightness intuition**

In a path graph with $\mathrm{diam}_G(K) = \Theta(|K|)$, any feasible $\kappa$ must have total capacity $\Omega(|K| \mathrm{diam}_G(K))$. Large diameter forces many vertices to become critical separators.

# A picture of critical vertices on a path



many middle vertices behave like separators

## Lesson

Capacity should concentrate on vertices that repeatedly act as separators. In low-diameter settings, the *total* required capacity stays small.

# Computing capacities by congestion balancing

## Conceptual congestion-balancing loop (Appendix A.1.5)

Initialize $\kappa(v) \leftarrow 1$ for all $v \in V(G)$. While there exists a sparse cut $L, S, R$ with respect to $K$ and $\kappa$:

- For each $v \in S$, update $\kappa(v) \leftarrow 2\kappa(v)$

Return $\kappa$.

## Caveat

Lemma I.2.9 is existential and the paper does *not* know how to compute its $\kappa$ in near-linear time in full generality. Instead, it computes a slightly relaxed capacity function (Lemma I.2.10) in time $\widehat{O}(n \cdot \mathrm{diam}_G(K))$, sufficient for the dynamic pruning used in Robust Core.

# From capacities to a maintained expander subset

## Capacitated pruning

Given $G$ and $\kappa$, apply capacitated expander pruning to maintain a large subset $X \subseteq K$ such that $G[X]$ remains a capacitated expander (with respect to $\kappa$) until significant total capacity is "deleted".

## Why this helps robust core

If $X$ remains large, then vertices staying within a moderate-radius ball around $X$ preserve a low-diameter core. When $X$ shrinks too much, recompute capacities and restart with a smaller terminal set.

## Important monotonicity invariant

Across restarts, capacities only increase. This yields a clean bound on the number of phases.

# Algorithm robust core (simplified) in phases

## Procedure RobustCore

```
Input graph G and initial terminal set K_init with diam_G(K_init) ≤ d
Set K ← K_init
while K is nonempty do
    Compute capacities κ (via Lemma I.2.10 style routine) so (K, κ) is a capacitated expander and Σ_v κ(v) = Ô(|K|d)
    Maintain expander subset X ⊆ K by capacitated pruning
    Maintain the ball B = ball_G(X, 4d)
    Remove from K every vertex that leaves B
    if |X| becomes too small then
        Start a new phase with the current K
    end if
end while
```

## Phase concept

A phase starts with a terminal set $K$ and ends when the maintained expander $X$ shrinks too much.

# Bounding the number of phases

## Cost per phase

Computing capacities and running pruning costs about $\widehat{O}(|K|d)$, which is $\widehat{O}(n)$ in the intended regime where $d = n^{o(1)}$ and degree is bounded.

## Why only few phases happen

A phase ends only after the adversary deletes a noticeable amount of total (edge) capacity inside the expander. But across phases the vertex capacities $\kappa(v)$ only increase and have bounded total sum. So only $\widehat{O}(1)$ phases can occur before the total deleted capacity exceeds the global bound.

## Monotonicity invariant

If $\kappa_{\text{old}}$ is from the previous phase and $\kappa_{\text{new}}$ is from the next phase, then

$$\kappa_{\text{old}}(v) \leq \kappa_{\text{new}}(v) \quad \text{for all } v.$$

# Approximate ball data structure

## Definition (Definition II.2.1)

An approximate ball structure $\mathrm{ApxBall}(G, S, D, \varepsilon)$ maintains estimates $\widetilde{d}(v)$ such that

- $\widetilde{d}(v) \geq \mathrm{dist}_G(S, v)$ for all $v$
- If $v \in \mathrm{ball}_G(S, D)$ then $\widetilde{d}(v) \leq (1+\varepsilon)\mathrm{dist}_G(S, v)$
- Each estimate only increases over time

## Abuse of notation

We also use $\mathrm{ApxBall}(G, S, D, \varepsilon)$ to denote the maintained set $\{v : \widetilde{d}(v) \leq (1+\varepsilon)D\}$.

# Maintaining a covering

## High level strategy

As long as some vertex is not covered by any existing core, create a new core around it. Each core is maintained by a robust core data structure. Around each core, maintain two approximate balls:

- A cover ball that ensures all vertices become covered
- A shell ball that upper bounds the work (shell overlap is bounded)

## Why multiple radii

Different cores are created with different radius scales $d^\ell$, supporting a smooth tradeoff between number of cores and shell overlap.

# Covering algorithm

## Core creation event

Pick an uncovered vertex $v$. Choose the smallest level $\ell$ such that ball growth slows:

$$|\mathrm{ball}_G(v, d^{\ell+1})| \leq n^{(\ell+1)/k}.$$

Set $C_{\mathrm{init}} = \mathrm{ball}_G(v, d^{\ell})$.

## Maintain three sets

- Core $C \subseteq C_{\mathrm{init}}$ via RobustCore on the shell graph
- Cover $\mathrm{cover}(C) = \mathrm{ApxBall}(G, C, 4d^{\ell}, \varepsilon)$
- Shell $\mathrm{shell}(C) = \mathrm{ApxBall}\left(G, C, \frac{\mathrm{str}}{4\varepsilon} d^{\ell}, \varepsilon\right)$

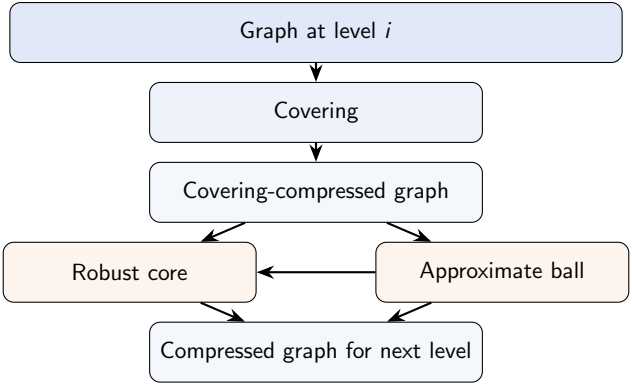
## Two key invariants

- Cores at the same level are disjoint
- Each vertex participates in only $n^{o(1)}$ shells overall

## Scattering as the accountant

When a vertex leaves a core, its local neighborhood at that level shrinks by a constant factor. So it cannot leave too many cores.

# Component diagram for Part II



# Problem statement for Part II

## Distance-only decremental SSSP

Maintain approximate distances from a fixed source $s$ in a weighted undirected graph undergoing edge deletions.

## Operations

- Update deletes an edge
- Query asks for an estimate $\widetilde{d}(v)$ of $\mathrm{dist}_G(s, v)$

## Performance goal

Total update time close to $m$ up to subpolynomial factors, with fast queries.

# Convenient assumptions via reductions

## Why reductions help

The full problem has arbitrary degrees, weight ratios $W$, and monotone weight increases. Part II reduces to a cleaner setting.

## Representative reduction statement (Proposition II.1.2)

It suffices to solve the problem for graphs that:

- Stay connected
- Undergo only edge deletions
- Have maximum degree three
- Have weights in a polynomial range (e.g. $[1, n^4]$)

# Layering across distance scales

## Multi-level design

Part II does not use just one scale: it maintains a hierarchy over distance thresholds $D_0 \leq D_1 \leq \cdots \leq D_{d_s}$.

## Depth of the hierarchy

The paper sets $d_s = \Theta(\log \log \log n)$ and chooses $D_i$ geometrically (roughly $(nW)^{i/d_s}$). Each level builds coverings/emulators appropriate to its distance range.

# Putting the components together

## At each layer

Maintain

- A covering of the current graph at that scale
- The covering-compressed graph
- An emulator $H_e$ constructed from the compressed graph
- An MES tree on $H_e$ to maintain distance estimates

# Main theorem revisited

**Theorem 4 (Theorem I.1.1)**

*For $\varepsilon > 1/\mathrm{polylog} n$ there is a deterministic decremental SSSP data structure in undirected weighted graphs that maintains $(1+\varepsilon)$ approximate distances from a fixed source with total update time*

$$m^{1+o(1)}\mathrm{polylog}\, W.$$

# Summary of the main ideas

## What enables adaptive decremental SSSP

- Reduce shortest path maintenance to maintaining hop emulators layer by layer
- Build hop emulators from coverings of low-diameter cores
- Solve the core stability problem via robust core
- Solve robust core via capacitated expanders and congestion balancing
- Package the result through compressed graph interfaces and MES trees

# References

A. Bernstein, M. P. Gutenberg, and T. Saranurak.
Deterministic decremental SSSP and approximate min-cost flow in almost-linear time.
In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1000–1008. IEEE, 2022.